

Evaluating LLMs

Measuring what you actually want

CS 5090 · Software Engineering with Generative AI · Week 8

Michigan Technological University · Fall 2026



Class 1 · What to measure, and how

Learning objectives

By the end of the week you should be able to:

1. **Measure generated code by running it**: explain why similarity is not correctness, and compute pass@k from n samples.
2. **Choose a way to get a metric** for an open-ended task and name its trade-off.
3. **Read a leaderboard critically**: contamination, weak tests, and Goodhart's law.
4. **Design an evaluation for a new capability**, with clarifying questions and code-optimizing agents as the examples.
5. **Check generated tests**: the oracle problem and mutation score.



Terms you will see this week

Benchmark: a fixed dataset plus a scoring rule, so numbers are comparable across models.

Reference (ground truth): the known correct answer a metric compares against.

Exact match: the output must equal the reference string.

BLEU: n-gram overlap between output and reference, 0–100 (from machine translation). **ROUGE**: recall version, for summaries. **BERTScore**: embedding similarity.

CodeBLEU: BLEU for code, adding syntax-tree and data-flow matching. Still a similarity score, not correctness.

Unit tests / hidden tests: run the code on inputs with known outputs; hidden = not shown to the model.

pass@k: fraction of problems solved by at least one of k samples.

Perplexity: exp(average per-token loss); the number of equally likely next tokens the model is choosing among.

Temperature: sampling randomness; 0 = greedy, always the most likely token.

Contamination: test data that leaked into the training data.

MMLU: 57-subject multiple-choice exam, auto-graded.
GSM8K: grade-school math, final number checked.

HumanEval / MBPP: short Python function tasks with unit tests (164 / ~1,000 problems).

HELM, Open LLM Leaderboard: an average over many auto-graded exams.

Chatbot Arena: two anonymous models answer the same prompt, the public votes; votes → a chess-style (Elo) rating.

Win rate: how often a model's answer is preferred over a baseline's, by a human or an LLM judge.

AlpacaEval: 805 instructions; a GPT-4 judge compares each answer with a baseline answer → win rate.

MT-Bench: 80 multi-turn questions; a GPT-4 judge scores answers 1–10.

LLM-as-a-judge: a strong model scores or compares outputs instead of a human.

Oracle: whatever decides what the right answer is (tests, a reference, a judge).

Baseline: what you compare against; a number without one is not a result.



Evaluation = dataset + metric

- Why evaluate: identify improvements, select models, decide if a system is ready for production.
- Classic AI: a closed-ended task with one known answer ("is this a dog?"). Easy to automate.
- LLMs: open-ended outputs, answers cannot be enumerated, quality is a continuum ("explain this bug").
- Six questions to ask of any evaluation: Is it cheap enough to run often? Does it measure what you actually care about? Does it separate models that really differ? Does it tell you *why* a model scored as it did? Will someone else get the same number? How hard is it to game? A product team weighs the last two most; a researcher the middle two.



Four ways to get a metric for an open-ended task

Approach	Example	Pro	Con
Convert to a closed-ended task	multiple choice (MMLU), final-answer match (GSM8K)	scalable, understandable	you changed the task
Reference-based heuristic	BLEU, ROUGE, BERTScore	same task	many valid answers, one reference
Humans	Chatbot Arena, expert raters	the evaluation you actually want	slow, costly, not reproducible
LLM as judge	AlpacaEval, MT-Bench	scalable and close to human	trust; needs a stronger "oracle" model

Code adds a fifth: execution (tests, compilers). The next five slides are about using it.



For code, similarity is not correctness

- Training loss measures next-token prediction; a model with low loss can still write code that does not compile.
- BLEU, CodeBLEU, and exact match compare tokens (CodeBLEU also syntax trees) with one reference. Two correct programs can share almost no tokens (for loop vs sum(...)); a wrong one can differ by one character (< vs <=).
- The figure: BLEU of correct (blue) and wrong (green) Codex solutions overlap on four HumanEval problems.
- The fix: **run it.** spec → model → code → hidden unit tests → pass / fail. The tests are the oracle.

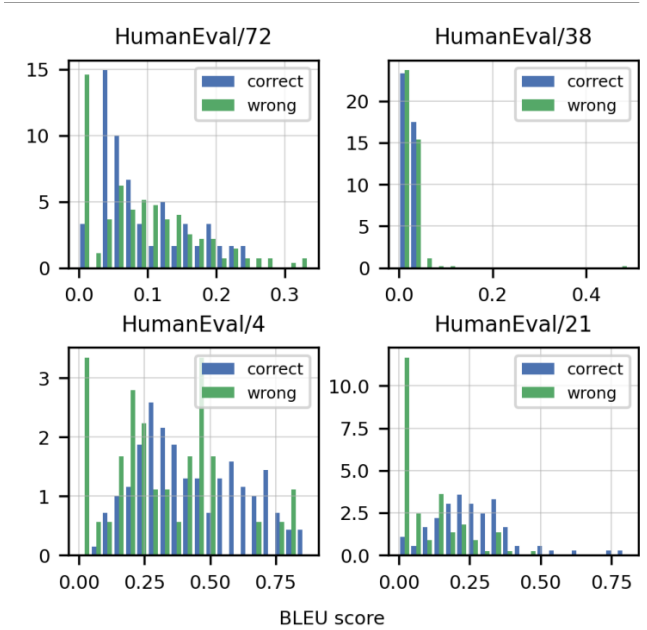


Figure: Chen et al. 2021, Fig. 8.

pass@k, revisited

- A problem counts as solved if **any** of k samples passes.
- Drawing exactly k samples once gives a noisy estimate.
- Chen et al. 2021: draw **n ≥ k** samples, count the **c** that pass, then (C(n, k) = "n choose k", the number of ways to pick k of n)

pass@k = 1 − C(n − c, k) / C(n, k), averaged over problems.



Worked example, and pass^k

One problem, **n = 10** samples, **c = 3** pass:

- pass@1 = 1 − C(7,1) / C(10,1) = 1 − 7/10 = **0.30**
- pass@5 = 1 − C(7,5) / C(10,5) = 1 − 21/252 ≈ **0.92**
- **pass^2** (did *both* of 2 attempts work?) = C(3,2) / C(10,2) = 3/45 ≈ **0.07**

Same samples, three numbers. pass@k is optimistic (something must pick the right one); pass@1 and pass^k are what a single run delivers. A tool you run once lives at pass@1.



Report the setup, not just the number

- Temperature (pass@1 at low temperature, pass@100 at high)
- n, k, number of problems, number of runs or seeds
- Prompt format and post-processing (stripping markdown, stop rules)
- Two papers reporting "pass@1 = 70%" can be measuring different things.



Perplexity: the intrinsic metric

- **PPL = exp(average per-token negative log-likelihood)**: the Week 4 loss, made interpretable.
- Between 1 and |V|. Read it as a **branching factor**: the number of equally likely next tokens the model is choosing among. On WikiText-103 the best models went from ~70 (2017) to under 10 (2023).
- Correlates with downstream performance, but depends on the data and tokenizer, so not comparable across models. Used in development, not as a benchmark.

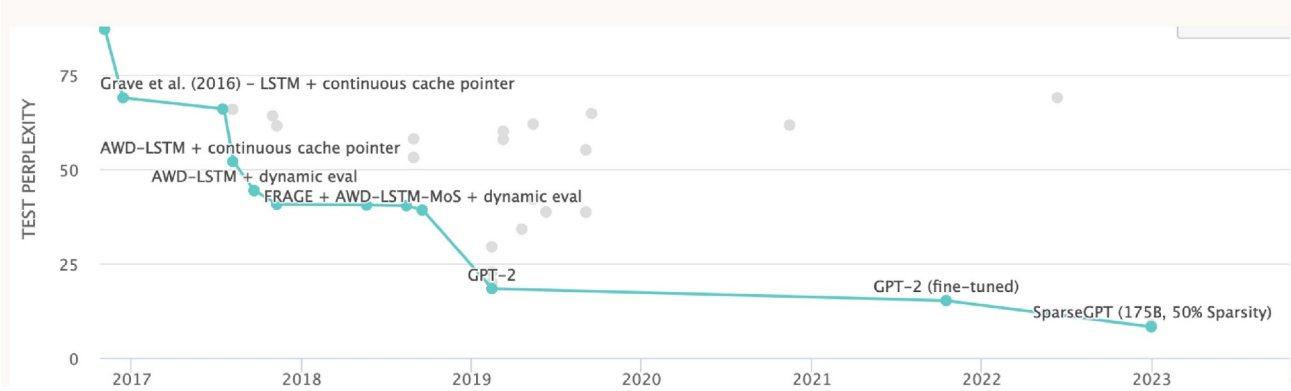


Chart: Papers with Code, WikiText-103 leaderboard, test perplexity over time.

How general LLMs are ranked

- **Exam averages**: many auto-graded exams, one mean score (HELM; Open LLM Leaderboard v2 = MMLU-Pro, GPQA, MATH, IFEval, BBH: knowledge, science, math, instruction following, reasoning). Simple and reproducible; does not evaluate open-ended generation; saturates, so the tasks get replaced.
- **Humans**: Chatbot Arena, blind pairwise votes from the public → an Elo-style rating. Realistic; slow; measures preference, not correctness.
- **LLM judges**: AlpacaEval (GPT-4 judge, win rate vs a baseline answer; ~0.94 correlation with Arena, under $10 per model), MT-Bench (GPT-4 judge scores 1–10). Scalable; needs an oracle; biased in known ways (Class 2).



Goodhart's law, with a real case

"When a measure becomes a target, it ceases to be a good measure."

- AlpacaEval's GPT-4 judge favours **longer** answers. Prompting the *same* model to be terse or verbose moved its win rate from **22.9% to 64.3%**; once the leaderboard mattered, models got wordier.
- Fix: a length-controlled win rate (regress the judge's preference on length, report it as if both answers were equally long). The same verbosity trick now moves the score only 41.9% → 51.6%, and agreement with Arena rises from 0.94 to 0.98.
- The same pattern elsewhere: training on test-like data (contamination), tuning the scaffold for one leaderboard, an agent editing the tests instead of the code (Week 5: reward hacking).
- Every metric has a shortcut; find it before the optimizer does.

	AlpacaEval			Length-controlled AlpacaEval		
	concise	standard	verbose	concise	standard	verbose
gpt4_1106_preview	22.9	50.0	64.3	41.9	50.0	51.6
Mixtral-8x7B-Instruct-v0.1	13.7	18.3	24.6	23.0	23.7	23.2
gpt4_0613	9.4	15.8	23.2	21.6	30.2	33.8
claude-2.1	9.2	15.7	24.4	18.2	25.3	30.3
gpt-3.5-turbo-1106	7.4	9.2	12.8	15.8	19.3	22.0
alpaca-7b	2.0	2.6	2.9	4.5	5.9	6.8

Figure: Dubois et al. 2024, Length-Controlled AlpacaEval, Fig. 3.



Key takeaways (Class 1)

- Evaluation = dataset + metric; pick the approach for your audience and know its trade-off.
- For code, execute the output. pass@k = 1 − C(n − c, k) / C(n, k); pass^k for reliability.
- Perplexity, exam averages, human preference, and LLM judges each measure something different.
- Goodhart: every metric has a shortcut.





Class 2 · Trusting the numbers, and making new ones

The code benchmarks you will see cited

Benchmark	What it is	Watch out for
HumanEval	164 hand-written Python functions with tests	small, saturated, few tests per problem
MBPP	~1,000 simple crowd-sourced Python tasks	similar shape and limits
HumanEval+ (EvalPlus)	HumanEval with many more tests	scores drop: the original tests missed bugs
SWE-bench Verified	real GitHub issues, judged by repo tests	public repos; the agent scaffold matters
LiveCodeBench	contest problems collected over time	designed against contamination



Weak tests inflate scores: EvalPlus

- EvalPlus (Liu et al. 2023) added many automatically generated tests to HumanEval; the result is HumanEval+.
- Solutions that passed HumanEval fail the stronger tests; every model's score drops, and rankings can change.
- Table: "base" is HumanEval, "+extra" is HumanEval+; GPT-4 drops from 88.4 to 76.2 pass@1.
- A benchmark score is bounded by the quality of its tests.

Table 3: Evaluating LLMs on HUMANEVAL and HUMANEVAL+. All models, except for INCODER, CodeGen2, StarCoder and SantaCoder which perform infilling, use auto-regressive generation. $k{=}1^{\star}$ marks pass@1 done with greedy decoding. T_k^* denotes the optimal pass@k temperature.

	Size	pass@k	$k{=}1^{\star}$	$k{=}1$	$k{=}10$	$k{=}100$	T_1^*	T_{10}^*	T_{100}^*
GPT-4 [49]	N/A	base	88.4						
		+extra	76.2						
Phind-CodeLlama [52]	34B	base	71.3	71.6	90.5	96.2	.2	.8	.8
		+extra	67.1	67.0	85.0	92.5	.2	.8	.8
WizardCoder-CodeLlama [38]	34B	base	73.2	61.6	85.2	94.5	.2	.8	.8
		+extra	64.6	54.5	78.6	88.9	.2	.8	.8
ChatGPT [48]	N/A	base	73.2	69.4	88.6	94.0			
		+extra	63.4	62.5	82.1	91.1			
CODELLAMA [54]	34B	base	51.8	52.0	82.4	95.0	.2	.8	.8
		+extra	42.7	43.1	73.7	89.4	.2	.8	.8
	13B	base	42.7	44.6	77.6	92.7	.4	.8	.8
		+extra	36.6	37.4	69.4	88.2	.4	.8	.8
	7B	base	37.8	39.2	69.1	89.7	.2	.8	.8
		+extra	34.1	34.5	61.4	82.9	.2	.8	.8
StarCoder [13]	15B	base	34.1	32.2	56.7	84.2	.2	.8	.8
		+extra	29.3	27.8	50.3	75.4	.2	.8	.8

Table: Liu et al. 2023 (EvalPlus), Table 3, excerpt.



Contamination

- Public benchmarks are on the web; the web is training data. Closed models cannot be audited.
- A memorized answer looks exactly like a solved problem. Symptom: strong on the benchmark, weak on fresh problems of the same kind.
- **Detect**: time split (before vs after the training cutoff, LiveCodeBench, figure); perturbation (EvoEval: the same problems rewritten by an LLM, scores drop if the model memorized the originals); likelihood tests (suspiciously high probability on test text; shuffle the examples and see if it drops).
- **Avoid**: private test sets, dynamic test sets that change over time.
- Two more reasons numbers differ: inconsistent setups (MMLU has several implementations with different prompts) and saturation.

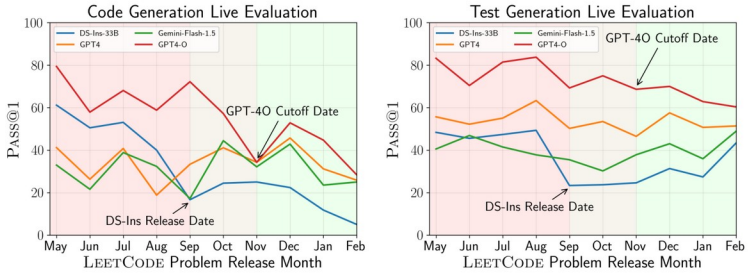


Figure: Jain et al. 2024, LiveCodeBench, Fig. 1.



Reading a contamination statement

Reading debrief: which leaderboard did you pick, and what does it say?

Ask three questions of any statement:

- Is there a cutoff date for the test problems, or a private set?
- Are the test cases hidden?
- Are the prompt, decoding, and scaffold reported?



LLM as a judge: when and why

- Many outputs cannot be executed: review comments, commit messages, explanations, clarifying questions.
- A strong model scores or compares outputs. Cheap and scalable; MT-Bench judges agree with humans about as often as humans agree with each other.
- Known biases: **position** (prefers the first answer), **verbosity** (prefers longer), **self-preference** (its own style). Fixes: swap the order, control for length, use a different model family as judge.

Table 2: Position bias of different LLM judges. Consistency is the percentage of cases where a judge gives consistent results when swapping the order of two assistants. "Biased toward first" is the percentage of cases when a judge favors the first answer. "Error" indicates wrong output formats. The two largest numbers in each column are in bold.

Judge	Prompt	Consistency	Biased toward first	Biased toward second	Error
Claude-v1	default	23.8%	**75.0%**	0.0%	1.2%
	rename	56.2%	11.2%	**28.7%**	**3.8%**
GPT-3.5	default	46.2%	**50.0%**	1.2%	2.5%
	rename	51.2%	38.8%	6.2%	**3.8%**
GPT-4	default	**65.0%**	30.0%	5.0%	0.0%
	rename	**66.2%**	28.7%	5.0%	0.0%

Table: Zheng et al. 2023 (MT-Bench), Table 2.



Make the judge accountable

- **Write the rubric before you look at any output**: a checklist, a reference solution, or a scoring guide. The judge only applies it. Agreement with experts goes up, and every score comes with a reason.
- **Calibrate**: label 50–100 items by hand and report judge–human agreement (percent agreement, or Cohen's kappa: agreement corrected for chance, 0 = chance, 1 = perfect). Low agreement means the judge's numbers mean nothing for your task.
- If your Project 2 uses a judge, the rubric and the calibration are part of the method.



Evaluating a new capability: should the model ask?

- Week 4: code models answer even when the problem is ambiguous, inconsistent, or incomplete; ClarifyCoder fine-tunes them to ask.
- pass@k cannot measure this: a clarifying question is not code, so it "fails" every test.
- **HumanEvalComm** (Wu & Fard, TOSEM 2025): take the 164 HumanEval problems, whose tests already exist, and modify each description to be ambiguous, inconsistent, or incomplete (762 problems). Now the right first response is a question, and the original tests still judge the final code.

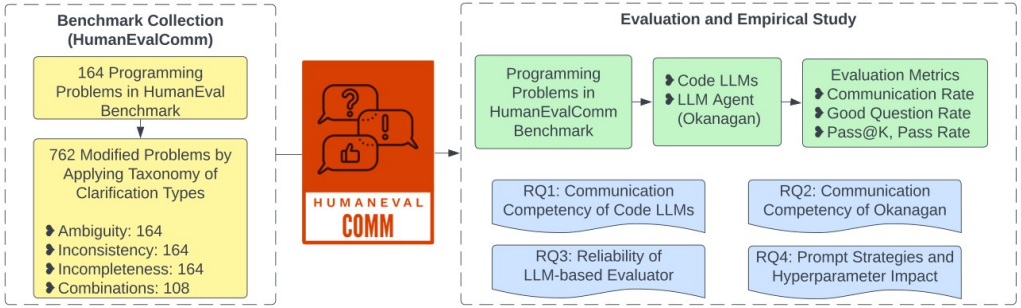


Figure: Wu & Fard 2025, HumanEvalComm, Fig. 1.



HumanEvalComm: metrics and what they showed

- **Communication Rate**: how often the model asks instead of writing code (closed-ended).
- **Good Question Rate**: how often the question is judged useful (LLM judge; the rubric-and-calibration rules apply).
- **Pass@1 and Test Pass Rate** after an LLM-simulated user answers (execution).
- Findings: most models rarely ask, so they confidently solve the wrong problem; models that ask more lose a little on well-specified problems. **Asking has a cost**; report both sides.
- Threats to validity to state out loud: the judge and the "user" are both LLMs. Lesson for Project 2: when you claim a new behavior, measure the behavior and what it costs.

Model	Pass@1		Test Pass Rate		Comm. Rate	Good Question Rate
	HmEval	*HmEvalComm*	*HmEval*	*HmEvalComm*		
ChatGPT	65.58%	31.34%	76.42%	49.39%	14.21%	13.43%
CodeLlama	29.88%	19.35%	45.71%	37.79%	10.16%	37.55%
CodeQwen1.5 Chat	76.83%	**47.61%**	84.4%	**62.89%**	4.82%	41.68%
DeepSeek Coder	71.78%	**45.68%**	79.44%	**62.25%**	**30.76%**	**61.42%**
DeepSeek Chat	12.8%	26.32%	13.86%	44.52%	**37.93%**	**58.71%**
Okanagan (Base=ChatGPT)	27.45%	**39.62%**	33.45%	**56.98%**	**72.73%**	**52.24%**
Okanagan (Base=DeepSeek	21.25%	**38.06%**	24.3%	**52.72%**	**82.51%**	**60.13%**

Table: Wu & Fard 2025, HumanEvalComm, Table 3.



GrowthHacker: evaluating agents that optimize code

A second new-capability example: the output is a number to improve, not code to pass tests (https://jie-jw-wu.github.io/blog/growthhacker.html)

- Task: off-policy evaluation, OPE (estimate how a new policy would do from logged data, instead of a live A/B test). Code from two open-source OPE libraries, OBP and Scope-RL (18 notebooks). The agent edits code or parameters, runs it, reads the error, repeats.
- Oracle: estimation error against a known ground truth (OBP: lower is better; Scope-RL: relative policy value, higher is better).
- Compared: a plain LLM (Gemini 2.5 Flash), two popular multi-agent frameworks (AutoGen, CrewAI), and a two-agent analyzer + coc

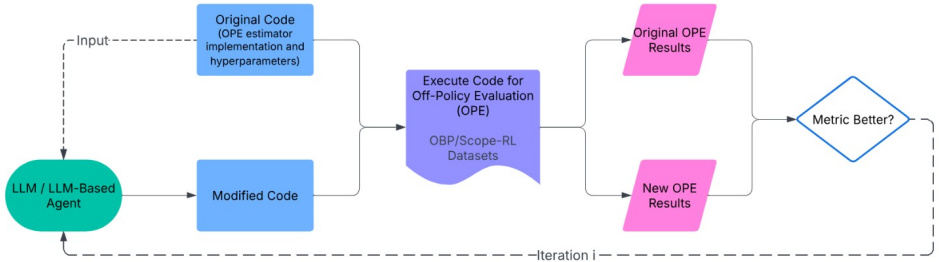


Figure: Wu et al. 2026, GrowthHacker,

The GrowthHacker workflow. Agents modify OPE code, execute the candidate, and compare its evaluation metric with the original implementation. *(click to enlarge)*



GrowthHacker: what the numbers say, and how to read them

- **Success rate** (does it run): two-agent 98–100%; general frameworks lower. Failures: syntax and runtime errors, agent control text leaking into code.
- **Outcome distribution**: positive / zero / negative / EXTREME. Two-agent 78% positive; AutoGen and Default 65%; CrewAI 63% but 0% EXTREME.
- **Size**: mean *and* median. CrewAI 37.9% vs 1.6% (a few huge wins); two-agent 26.0% vs 4.4%.
- **Baseline**: random search in the same bounds matches the LLMs on one estimator but completes only 56% of Scope-RL runs.

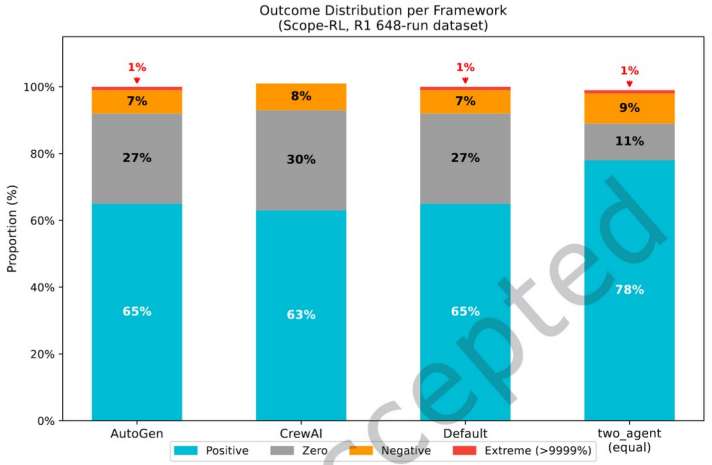


Fig. 5. Stacked bar chart showing normalized outcome proportions per framework (Scope-RL, R1 648-run dataset). Each bar sums to 100%.

Accepted

Figure: Wu et al. 2026, GrowthHacker, Fig. 5.



⚙️ Testing with LLMs: the oracle problem

- Tests are the most reliably useful thing LLMs write: tedious, well specified, mechanically checkable.
- The trap: your function has an off-by-one bug; the model reads the code and writes a test asserting what the code **does**; the test passes ✓; the bug is now "tested."
- A test generated from the implementation checks behavior, not intent. The oracle must come from the **specification**.
- Three ways out: write the test first (TDD, test-driven development); generate tests from the spec, not the code; let the model write inputs and setup, but **keep the assertions yours**.



Coverage vs mutation score

- Coverage measures which lines **ran**; a suite with no assertions can reach 100% and catch nothing.
- Mutation testing: inject small bugs into the code (< → <=, + → -); each buggy copy is a **mutant**, and it is **killed** if some test fails on it.
- **Mutation score = killed / total mutants.** Example: 10 mutants, 7 killed → 70%; the 3 survivors show where assertions are missing.
- Tools: PIT (Java) and mutmut (Python) generate the mutants and run your suite against them.



Your Project 2 evaluation

For the proposal (due Sun, Nov 8):

- State the claim in one sentence, then the metric that tests it, and which of the four approaches it uses.
- Choose a baseline you could honestly lose to.
- Fix n, k, temperature, and the number of runs; report variance.
- Public benchmark → say how you handled contamination. LLM judge → rubric plus calibration. New behavior → measure it and its cost.



Key takeaways, Module 1 wrap, deadlines

- A benchmark score is bounded by its tests and by contamination; ask for the cutoff, the hidden tests, and the scaffold.
- Judge with a rubric and calibrate against humans.
- A new capability needs its own metrics, plus the cost it introduces.
- A test written from the code cannot catch a bug in that code; mutation score measures what the tests would catch.
- Module 1: tokens → Transformer → pretraining → fine-tuning → preferences and reasoning → retrieval and agents → **evaluation**. Next: Module 2, the model as one component of a software system.
- Project 1 due in full Fri, Oct 30 · Project 2 proposal due Sun, Nov 8.



References

Austin, J. et al. 2021. Program Synthesis with Large Language Models (MBPP). arXiv:2108.07732.

Barr, E. T. et al. 2015. The Oracle Problem in Software Testing: A Survey. IEEE TSE 41(5).

Chen, M. et al. 2021. Evaluating Large Language Models Trained on Code (HumanEval, pass@k). arXiv:2107.03374.

Chiang, W.-L. et al. 2024. Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference. ICML 2024.

Cohen, J. 1960. A Coefficient of Agreement for Nominal Scales. Educational and Psychological Measurement 20(1).

Dubois, Y. 2024. Scalable Evaluation of LLMs. Lecture slides, CIS 7000, University of Pennsylvania.

Dubois, Y. et al. 2024. Length-Controlled AlpacaEval: A Simple Way to Debias Automatic Evaluators. arXiv:2404.04475.

Fourrier, C. et al. 2024. Open LLM Leaderboard v2. Hugging Face.

Hendrycks, D. et al. 2021. Measuring Massive Multitask Language Understanding (MMLU). ICLR 2021.

Jain, N. et al. 2024. LiveCodeBench: Holistic and Contamination Free Evaluation of LLMs for Code. arXiv:2403.07974.

Jia, Y. and Harman, M. 2011. An Analysis and Survey of the Development of Mutation Testing. IEEE TSE 37(5).

Jimenez, C. E. et al. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024.

Liang, P. et al. 2022. Holistic Evaluation of Language Models (HELM). arXiv:2211.09110.

Liu, J. et al. 2023. Is Your Code Generated by ChatGPT Really Correct? (EvalPlus). NeurIPS 2023.

Oren, Y. et al. 2024. Proving Test Set Contamination in Black Box Language Models. ICLR 2024.

Strathern, M. 1997. "Improving ratings": audit in the British University system. European Review 5(3).

Wu, J. J. W. and Fard, F. H. 2025. HumanEvalComm: Benchmarking the Communication Competence of Code Generation for LLMs and LLM Agent. ACM TOSEM. arXiv:2406.00215.

Wu, J. J. W. et al. 2025. ClarifyCoder: Clarification-Aware Fine-tuning for Programmatic Problem Solving. arXiv 2025.

Wu, J. J. W., Herlihy, A. P., Mirza, A. S., Afoud, A. and Fard, F. H. 2026. GrowthHacker: Automated Off-Policy Evaluation Optimization Using Code-Modifying LLM Agents. ACM TOSEM. arXiv:2511.00802.

Xia, C. S. et al. 2024. Top Leaderboard Ranking = Top Coding Proficiency, Always? EvoEval. arXiv:2403.19114.

Yao, S. et al. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains (pass^k). arXiv:2406.12045.

Zheng, L. et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023.

Grounding & Evaluation for LLMs, KDD 2024 tutorial slides (RAG evaluation dimensions).



Acknowledgements

This deck draws on the sources below (full entries on the References slide).

- **Yann Dubois, Scalable Evaluation of LLMs (CIS 7000, University of Pennsylvania, 2024).** The organizing idea of Class 1 comes from these lectures: the "dataset + metric" framing, the four ways to obtain a metric and their trade-offs, the treatment of perplexity, the exam-average / human / LLM-judge taxonomy of leaderboards, the AlpacaEval length-bias case, and fixing a rubric before judging. Slides 5, 6, 10, 12, 13 and 21 are rewritten from that outline.
- **Grounding & Evaluation for LLMs, KDD 2024 tutorial.** Background on general-LLM evaluation dimensions.
- **Chen et al. 2021 (Codex / HumanEval).** The pass@k estimator, the worked example, the BLEU-vs-correctness figure, and the temperature discussion.
- **Liu et al. 2023 (EvalPlus); Jain et al. 2024 (LiveCodeBench); Xia et al. 2024 (EvoEval); Oren et al. 2024.** The weak-tests and contamination material of Class 2.
- **Zheng et al. 2023 (MT-Bench); Dubois et al. 2024 (Length-Controlled AlpacaEval); Chiang et al. 2024 (Chatbot Arena).** LLM-as-a-judge, its biases and fixes, and human-preference evaluation.
- **Wu & Fard 2025 (HumanEvalComm); Wu et al. 2026 (GrowthHacker); Wu et al. 2025 (ClarifyCoder).** The two worked examples of evaluating a new capability; the instructor is an author on these.
- **Barr et al. 2015; Jia & Harman 2011; Strathern 1997; Cohen 1960.** The oracle problem, mutation testing, Goodhart's law as quoted, and Cohen's kappa.
- Reproduced figures and tables carry a source line on their slide; all are used for classroom teaching under fair use.

